

Applicazioni: Cicli Hamiltoniani in grafi Aleatori

Si può far vedere come il problema del ciclo Hamiltoniano possa diventare facile su una classe di grafi come quelli Aleatori.

Introduciamo ora 2 famose classi di grafo aleatorio.

Def

nel No e $pe(2,1)$

Il modello di Erdős-Rényi, indicato $G_{n,p}$ è un grafo aleatorio su n vertici $v_1, \dots, v_n$ in cui ognuno dei possibili $\binom{n}{2}$ archi è presente indipendentemente con prob. p .

Oss

- La prob. di osservare un dato grafo con esattamente m archi in $G_{n,p}$ è $p^m (1-p)^{\binom{n}{2}-m}$
- In $G_{n,p}$ ci sono in media $\binom{n}{2} p$ archi ed ogni vertice ha un grado atteso di $(n-1)p$.

Def

con $n, N \in \mathbb{N}$

Il modello $G_{n,N}$ è un grafo Aleatorio su n vertici $v_1, \dots, v_n$ con esattamente N archi.

Degli $\binom{n}{2}$ possibili grafi se ne sceglie ciascuno con uguale prob.

Oss

I due modelli sono strettamente legati:

Infatti se condizioniamo $G_{n,p}$ ad avere N archi allora otteniamo un $G_{n,N}$.

Inoltre prendendo $p = \frac{N}{\binom{n}{2}}$ il numero di archi in $G_{n,p}$ si concentra fortemente attorno a N .

La relazione è simile a quella tra il modello delle urne e palline e quello con le n Poisson indipendenti.

Ci concentreremo ora su $G_{n,p}$ e faremo vedere che con alta prob. è facile trovare un ciclo Hamiltoniano su tale grafo se p è sufficientemente grande.

Attenzione!

Con alta prob. si intende sia la prob. legata al modello $G_{n,p}$ di selezionare un dato grafo, sia all'aleatorietà introdotta dall'algoritmo che utilizzeremo...

Scegliamo per un istante di $G_{n,p}$ ed introduciamo un algoritmo per trovare cicli Hamiltoniani su un dato grafo. Ricordiamo che

Def

Un cammino Hamiltoniano in un grafo è un cammino che attraversa ogni vertice esattamente una volta.

Un ciclo Hamiltoniano è un cammino chiuso.

Introduciamo un'operazione su un cammino detta rotazione.

Def

Sia $P = v_1, \dots, v_k$ un cammino semplice su un grafo non orientato G e sia (v_k, v_i) un arco di G .

Allora il cammino detto Rotazione di P attraverso (v_k, v_i) è il cammino semplice

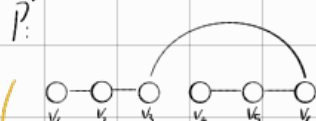
$$P' = v_1, v_2, \dots, v_i, v_k, v_{k-1}, \dots, v_{i+1}, v_i$$

Esempio

P :



P' :



Rotazione di P attraverso (v_3, v_6)

Ecco un primo algoritmo per cercare un ciclo Hamiltoniano in un grafo.

L'input è rappresentato da n liste, ogni lista corrisponde alla lista di archi che partono da ciascuno degli n vertici ordinati in modo casuale.

All'inizio l'algoritmo sceglie un nodo a caso, la "testa" del cammino.

Ad ogni passo l'algoritmo o aggiunge un nuovo vertice al cammino, facendolo diventare la nuova testa oppure effettua una rotazione.

L'algoritmo si ferma quando la lista della testa diventa vuota.

Algoritmo per trovare un ciclo Hamiltoniano

Input: un grafo $G=(V,E)$ con n nodi

Output: un ciclo Hamiltoniano o fallimento

- 1) Selezionare un vertice a caso, la testa del cammino
 - 2) Ripetere i seguenti passi fino a che l'arco della rotazione non chiude un ciclo Hamiltoniano o fino a che la lista di archi inutilizzati della testa si svuota:
 - a) Sia $P=v_1, \dots, v_k$ il cammino attuale con v_k la testa e sia (v_k, u) il primo arco della lista della testa
 - b) Cancellare (v_k, u) dalle liste di v_k e di u
 - c) Se $u \neq v_1$ per qualche $i=1, \dots, k-1$, aggiungere $u=v_{k+1}$ al cammino e farlo diventare la nuova testa
 - d) Se invece $\exists i \in \{1, \dots, k-1\}$ t.c. $u=v_i$, ruotare il cammino corrente P all'indietro (v_k, v_i) e porre v_{i+1} come la nuova testa
- Questo passo chiude un ciclo Hamiltoniano se $k=n$ e sceglie (v_n, v_1)
- 3) Restituire il ciclo Hamiltoniano se e' stato trovato, oppure fallimento

Questo semplice algoritmo e' difficile da analizzare a causa delle dipendenze che si creano

una volta che gli archi sono stati esplorati, la distribuzione degli archi ancora da esplorare dipende da essi

Per risolvere ora l'algoritmo per evitare questo problema, aggiungendo alcuni trucchi

Nota

L'algoritmo precedente, una volta scelto l'ordine delle liste di ogni nodo e la testa iniziale, procedeva in maniera del tutto deterministica

Il nuovo algoritmo e' meno efficiente, ma piu' facile da analizzare

Ora ogni vertice v avra' due liste attaccate:

gli $\text{ArchiUsati}(v)$ che contiene gli archi adiacenti a v usati dall'algo mentre v era la testa

gli $\text{ArchiNonUsati}(v)$ con all'interno quelli ancora mai utilizzati

Algoritmo modificado per il ciclo Hamiltoniano

Input: un grafo $G=(V,E)$ con n nodi

Output: un ciclo Hamiltoniano o fallimento

1) Selezionare un vertice a caso, la testa del cammino

2) Ripetere i seguenti passi sino a che l'arco della rotazione non chiude un ciclo Hamiltoniano o sino a che la lista

$\text{ArchiNonUsati}(testa)$ e' vuota:

a) Sia $P=v_1, \dots, v_k$ il cammino attuale con v_k la testa

b) Eseguire

[i] con prob. $\frac{1}{n}$

[ii] con prob. $\frac{|\text{ArchiUsati}(v_k)|}{n}$

[iii] con prob. $1 - \frac{1}{n} - \frac{|\text{ArchiNonUsati}(v_k)|}{n}$

[i] Inventare il cammino e rendere v_i la nuova testa

[ii] Scegliere un i a caso un arco dalla lista $\text{ArchiUsati}(v_k)$:

se l'arco e' (v_k, v_i) per qualche $i=1, \dots, k-1$, ruotare il cammino P tramite (v_k, v_i) e rendere v_{i+1} la nuova testa, Altrimenti non fare nulla

[iii] Scegliere il i -esimo della lista $\text{ArchiNonUsati}(v_k)$, sia (v_k, u)

se $u \neq v_1$, $\forall i=1, \dots, k$, aggiungere $u=v_{k+1}$ al cammino e rendere v_{k+1} la nuova testa

se invece $\exists i$ tale che $u=v_i$, ruotare il cammino corrente P attraverso (v_k, v_i) e rendere v_{i+1} la nuova testa

→ Questo passo chiude il cammino se $k=n$ e (v_n, v_1) viene scelto

3) Restituire il ciclo Hamiltoniano se e' stato trovato, oppure fallimento

Obiettivo

Vogliamo dimostrare che quest'algoritmo trova un ciclo Hamiltoniano in un grafo campionato secondo il modello Erdős-Rényi $G_{n,p}$ quando p e' sufficientemente grande con alta prob.

Prima di trattare G_{np} analizziamo l'algo assumendo una particolare distribuzione iniziale degli ArchiNonUsati :

Assumiamo che $\forall$ vertice v , ciascuno dei possibili $n-1$ archi che connettono v ad un'altra vertice sia nella lista $\text{ArchiNonUsati}(v)$ all'inizio con prob $q \in (0,1)$

Assumiamo inoltre che in ogni lista l'ordine sia casuale

oss

In questo modo potrebbe capitare che un arco (u,v) sia inizialmente in $\text{ArchiNonUsati}(v)$ ma non in $\text{ArchiNonUsati}(u)$

In questo modello, se v e' la testa e l'arco (u,v) viene selezionato per la prima volta, allora (u,v) viene rimosso da $\text{ArchiNonUsati}(v)$ ma non da $\text{ArchiNonUsati}(u)$ $\rightarrow$ se presente

Nota

Cosa abbiamo guadagnato aggiungendo aleatorietà nel passo 2 dell'algoritmo?

Faremo vedere che in questo modo ad ogni passo la nuova testa del cammino sarà scelta in maniera uniforme tra gli n vertici

Sottolineando moltissimo l'analisi dell'Algo $\rightarrow$ riducendolo ad un Coupon collector

Lemma 5.16

Utilizzando l'Algo modificato su questo nuovo modello si verifica la cosa seguente:

Sia v_t la testa del cammino al t -esimo passo dell'algoritmo.

Allora per ogni vertice u , se al t -esimo passo $\text{ArchiNonUsati}(v_t)$ e' non vuoto, si ha che:

$$P(v_{t+1} = u | v_t = u, v_{t-1} = u_{t-1}, \dots, v_0 = u_0) = \frac{1}{n}$$

Cioe' la testa del cammino può essere considerata come presa a caso ad ogni passo, indipendentemente da quanto successo sino a quel punto

Dim

Consideriamo tutte le possibilità se $P = v_1, \dots, v_t$

$$\bullet P(v_{t+1} = v_i | \text{Al tempo } t \text{ ho } P) = \frac{1}{n}$$

caso b[i]

poiche' l'unico modo in cui v_i possa diventare la nuova testa e' che il cammino P venga invertito

$$\bullet \mathbb{P}(V_{t+1} = v_{i+1} \mid \text{Al tempo } t) =$$

$$= \mathbb{P}(V_{t+1} = v_{i+1} \cap (v_k, v_i) \in \text{ArchiUsati}(v_k) \mid \text{Al tempo } t) + \mathbb{P}(V_{t+1} = v_{i+1} \cap (v_k, v_i) \notin \text{ArchiUsati}(v_k) \mid \text{Al tempo } t)$$

La 1^a delle due vale $\frac{|\text{ArchiUsati}(v_k)|}{n} \cdot \frac{1}{|\text{ArchiUsati}(v_k)|} = \frac{1}{n}$

caso b[iii]

La 2^a invece $\left(1 - \frac{1}{n} - \frac{|\text{ArchiNonUsati}(v_k)|}{n}\right) \left(\frac{1}{n - |\text{ArchiUsati}(v_k)| - 1}\right) = \frac{1}{n}$

caso b[iii]

All'interno di $\text{ArchiUsati}(v_k)$ vi sono archi scelti in modo indep. dalle liste degli altri vertici e sono stati messi in ordine casuale. Possiamo pensare che ogni volta che l'algo. sceglie [iii] un nuovo arco della lista $\text{ArchiNonUsati}(v_k)$ viene rivelato e siccome ogni arco (v_k, v) che non è stato usato sino a quel momento ha uguale probabilità di essere in cima alla lista $\text{ArchiNonUsati}(v_k)$ tra gli $n - 1 - |\text{ArchiUsati}(v_k)|$ disponibili, si ha che la prob. che venga scelto è esattamente quella indicata

Per cui abbiamo quindi ottenuto che

$$\mathbb{P}(V_{t+1} = v_{i+1} \mid \text{Al tempo } t) = \frac{1}{n}$$

• Infine, se $\mathcal{I}_i: u = v_i$ si ha

$$\mathbb{P}(V_{t+1} = u \mid \text{Al tempo } t) = \frac{1}{n}$$

poiché questa è la probabilità che (v_k, u) venga rivelato come prossimo elemento della lista $\text{ArchiNonUsati}(v_k)$, e si ragiona come prima

Nota

Grazie a questo Lemma abbiamo ridotto il problema ad uno del tipo "urne e palline" o "collezionatore di figurine"

Infatti, quando il cammino è lungo $n-k$, la prob. di allungarlo di un vertice è $1/k$, cioè la prob. che l'Algo scelga uno dei k vertici rimanenti

Una volta che il cammino è lungo n , la prob. di chiuderlo in un ciclo Hamiltoniano è $1/n$.

Perciò avremo

Sono reslando che la lista $\text{ArchiNonUsati}(v_k)$ non si

• $O(n \log n)$ passi per formare un ciclo Hamiltoniano

Sia mai vuota

• $O(n \log n)$ ulteriori passi (rotazioni) per chiuderlo in un ciclo Hamiltoniano

Thm 5.17

Supponiamo di avere come input dell'Algo modificate liste di ArchiNonUsati in cui ogni arco (v, u) , con $v \neq u$, si trova in $\text{ArchiNonUsati}(v)$ indipendentemente con prob. $q \geq \frac{2 \log n}{n}$

Allora l'Algo trova un ciclo Hamiltoniano in

$O(n \log n)$ passi con prob. $1 - O(\frac{1}{n})$

Dim

Consideriamo i due eventi:

E_1 = "In $3n \log n$ passi nessuna lista di ArchiNonUsati si svuota e l'Algo non trova un ciclo Hamiltoniano"

E_2 = "Almeno una lista di ArchiNonUsati si svuota in $3n \log n$ passi"

Si ha che l'Algo Fallisce $\Leftrightarrow E_1$ o E_2 si realizzano

Ci basta far vedere che sia E_1 che E_2 hanno prob. bassa

• E_1 :

Dal Lemma 5.16 sappiamo che se le liste di ArchiNonUsati non sono vuote, ad ogni passo l'Algo sceglie una nuova lista in modo univ. fra gli n vertici.

Condizionatamente a liste non vuote si ha allora:

$$\begin{aligned} \mathbb{P}(\text{In } 3n \log n \text{ passi non trovo un H.P. lungo } n) &= \\ &= \mathbb{P}(\text{Il coupon collector deve aprire più di } n \log n \text{ pacchetti per trovare } n \text{ figurine differenti}) \leq \\ &\leq \sum_{i=1}^n \mathbb{P}(\text{i-esima figurina Non-trovata in } 2n \log n \text{ pacchetti}) \leq \\ &\leq \sum_{i=1}^n \left(1 - \frac{1}{n}\right)^{2n \log n} \leq n e^{-\frac{2}{n}(n \log n)} = \frac{1}{n} \end{aligned}$$